

ONN implementation of FPGA for digit recognition

Chinmay Kulkarni

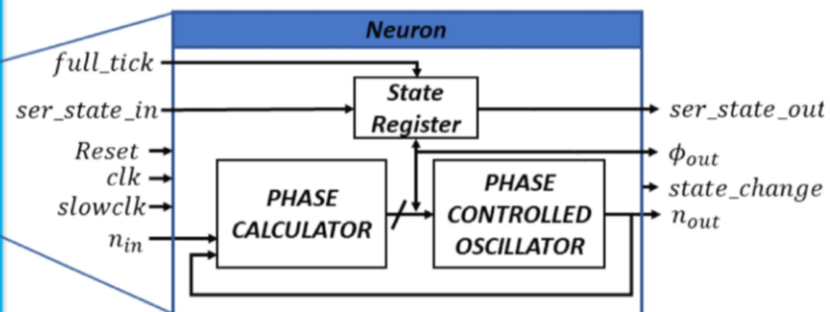
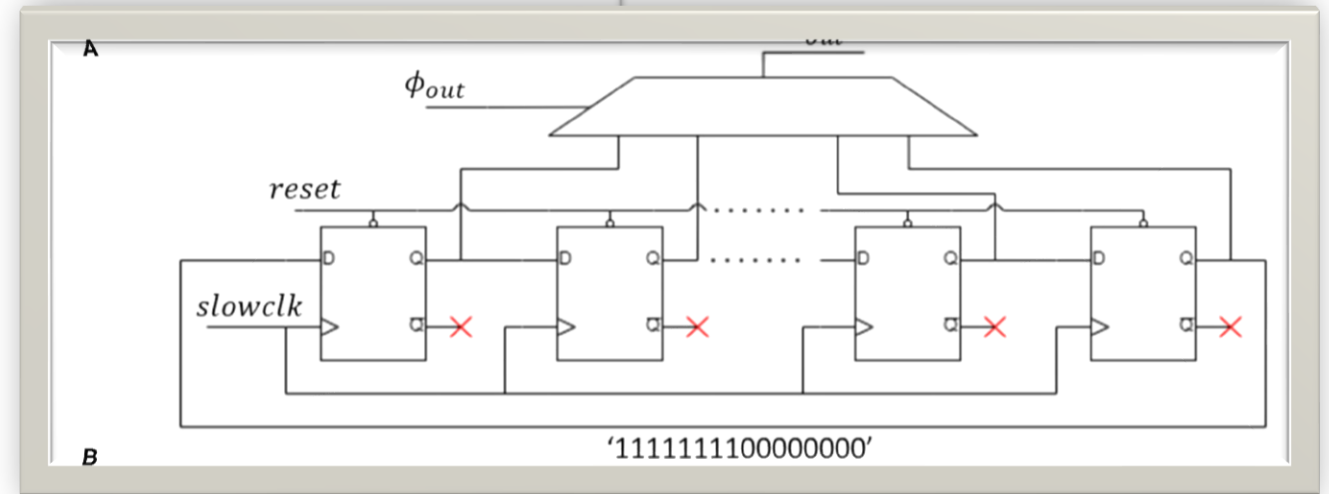
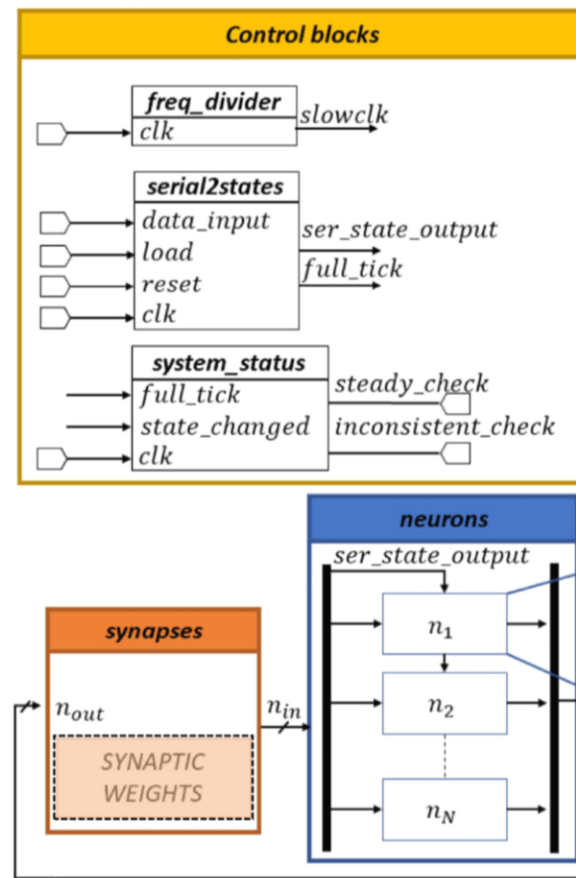
Prof. Sandip Lashkare

Introduction:-

- **Objective:** To recognise handwritten digits from images using an ONN.
- **Advantages of Oscillatory Neural Networks (ONNs):**
 - Low power consumption
 - Memory-efficient computation

ONN architecture

DIGITAL ONN



Work done on the project in the last semester

- Most of the time is utilised in learning Verilog and understanding ONN digital design
- I have implemented ONN for a small image of size 5x3 for digits 0,1,2 only
- Even when parts of the digit were missing or distorted, the ONN could still co



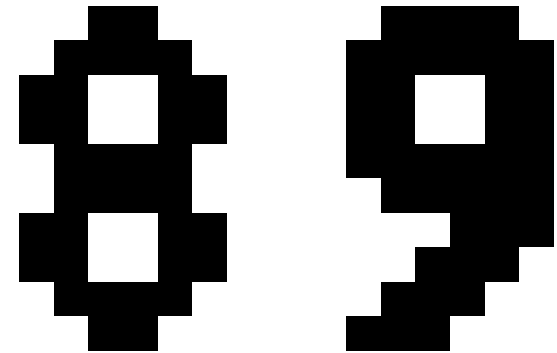
Work done in may

- Now, after successfully implementing ONN for a 5x3 image, I decided to work with a bigger image of size 10x6
- I created a custom dataset of all 10 digits and hand-drawn distorted images for training weights
- Outputs were heavily biased towards the digit 9.



Solution:-

- As I have the liberty to choose the digit image, I decided to change the image of 8,9 to get rid of the bias.
- Also trained weights using new images.
- Got the results for base image detection successfully.



Moving to real life application

- The work done till now was based on self-made images, which will not work in real life
- Now I have started working on the EMNIST dataset.
- The original EMNIST data set has images of size 28X28 pixels,
 - Which can't be used as it will overflow the hardware limitations of the FPGA board.
- So I choose to work on smaller images.
- Also, just for digits 0,1.

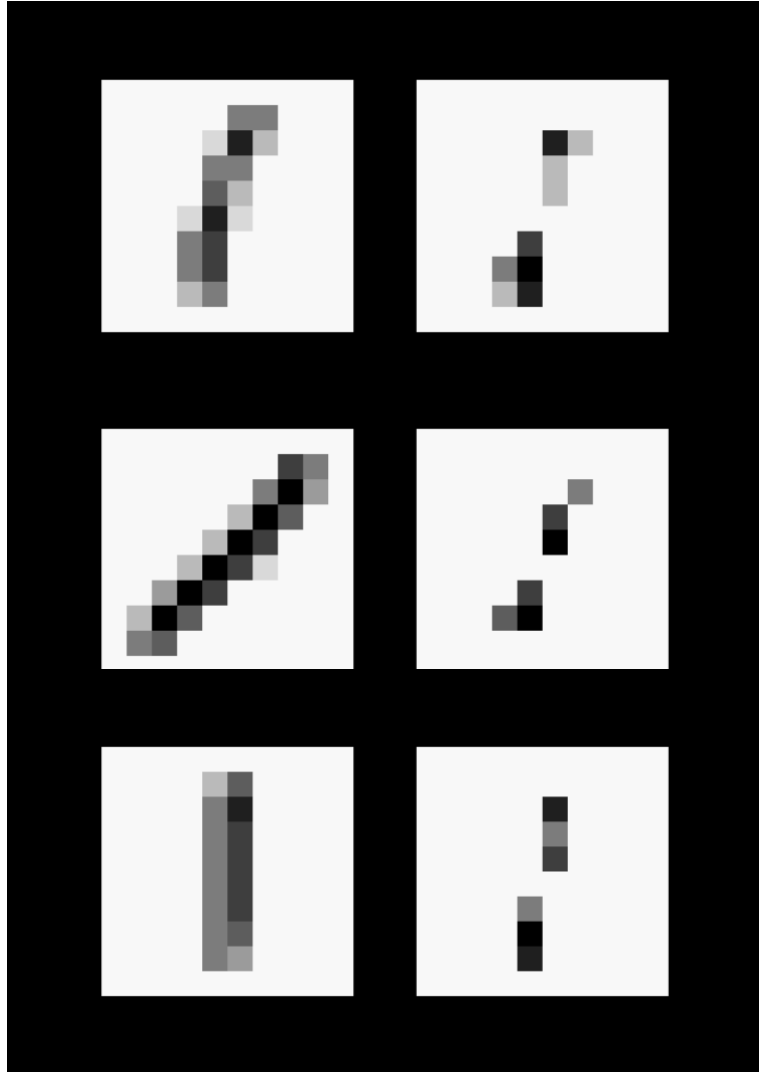
Work with 14X14 image

- I have started working with 14x14-sized images.
- But I have realized that it will also overflow hardware constraints
So I moved to a much smaller image, 10x10.
- I converted 28x 28 size image to 10x10 by average pooling algorithm
In Python.

Work with 10X10 images(random images)

- ☐ I have got random images of 1,0 from EMNIST dataset.
- ☐ Trained weights with them and clipped them to size of 5 bits and 9 bits, and tested simulations on ONN.

Results with 5-bit weights

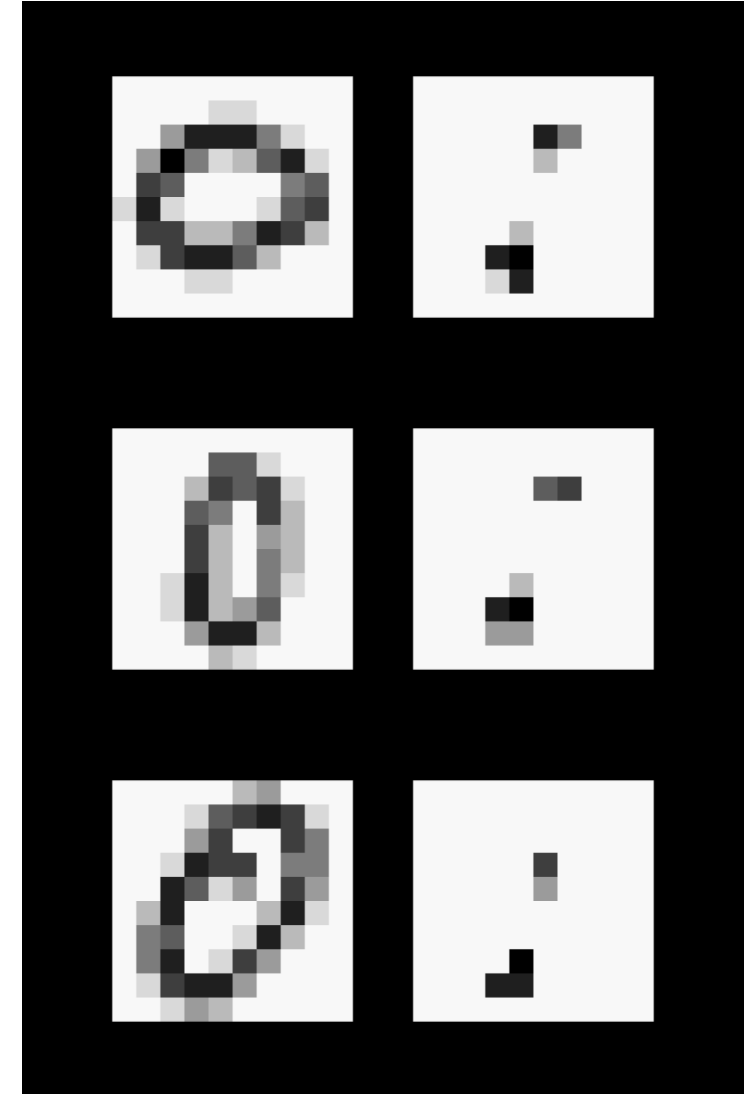


Problems:

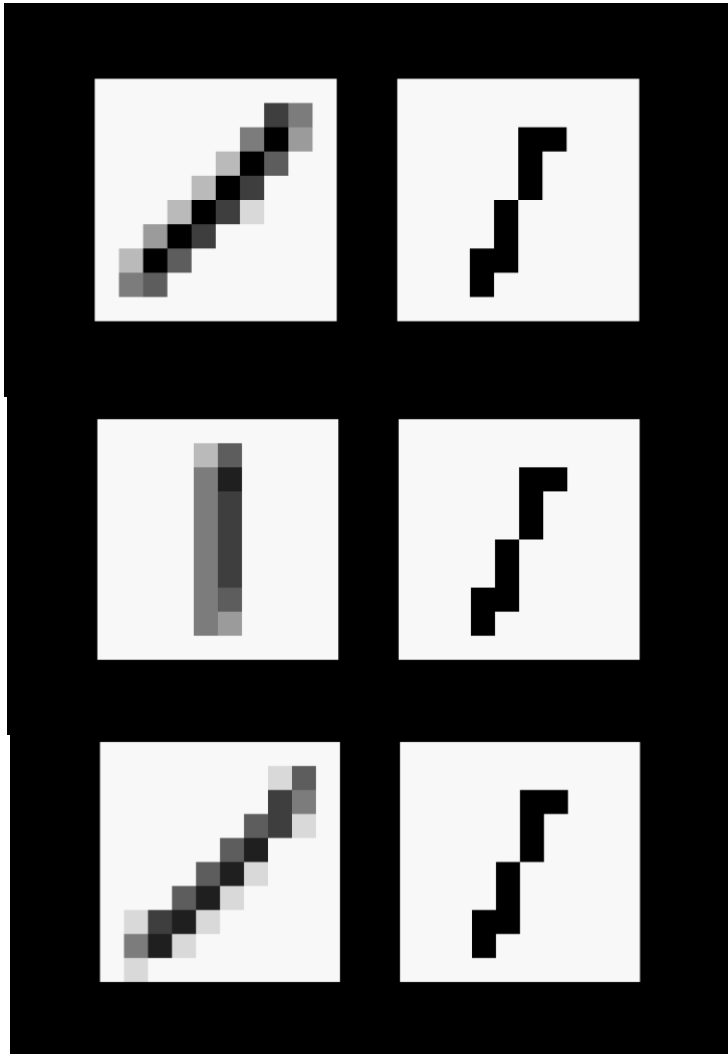
1. Different shapes and sizes of digits are causing errors in the output .

Solution I tried

- Changed weight size to 9 bits.



Results with 9bit weights

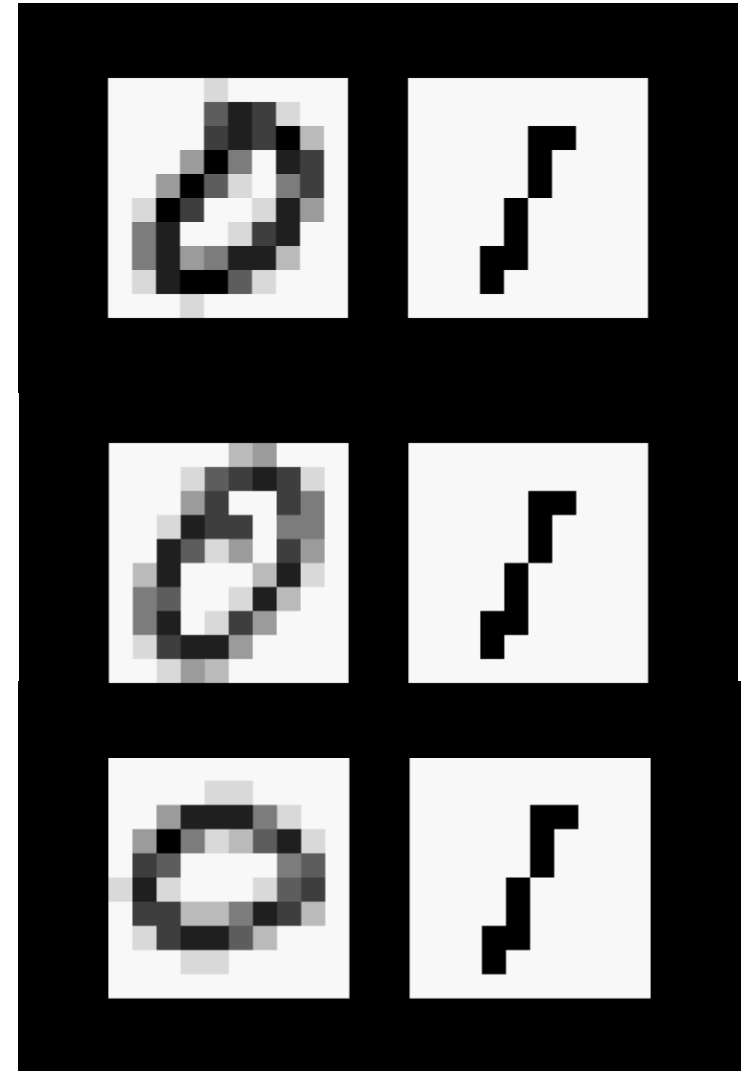


Problems:

- 9-bit weights cause similar output at all times.

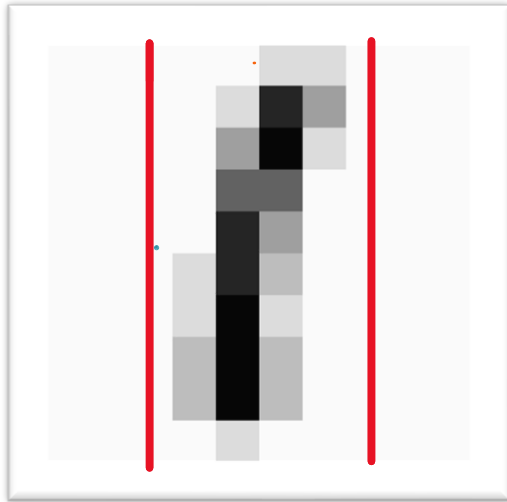
Solution I tried

- Constrained images to a particular shape to get good weights.
- Further, I haven't trained any weights of 9 bits as it will cause hardware constraints.

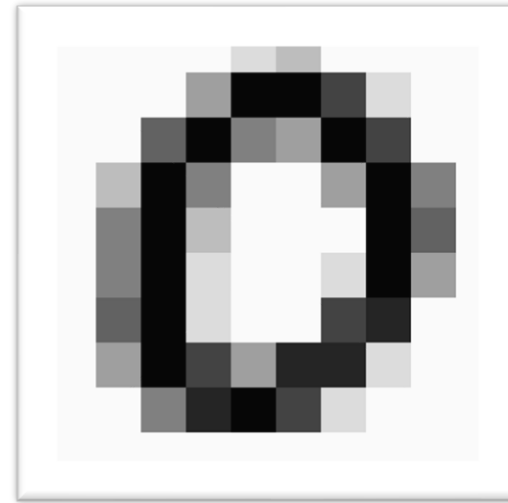


Work with constrained images

✦ I have worked on the following types of constrained images

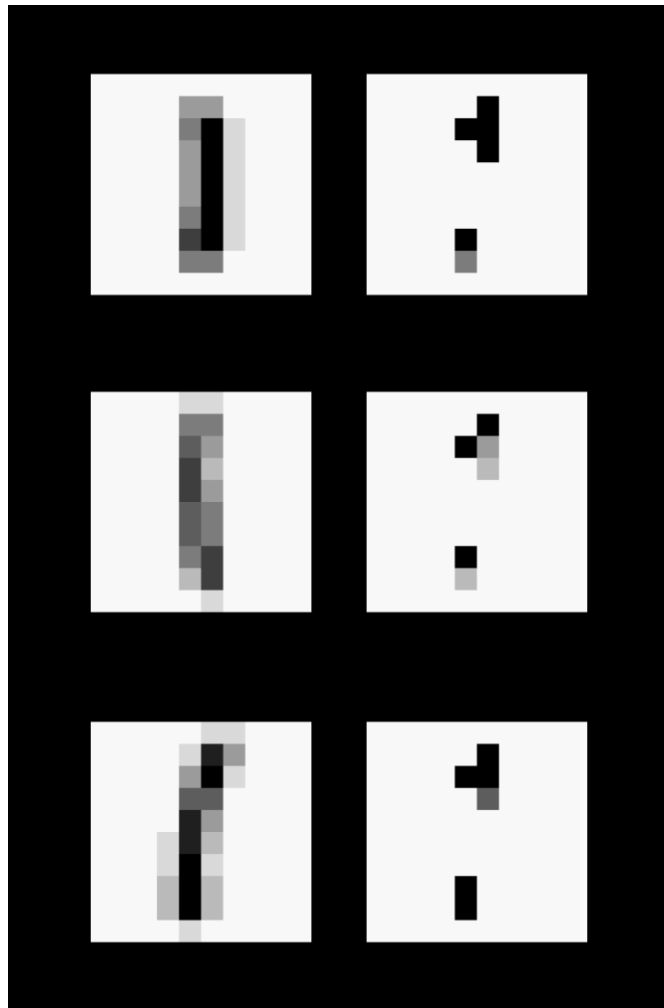


- A constrained digit "1" is positioned approximately in the middle of the image.
- It is also conveniently straight.



- Constrained images of 0 should be approximately perfect circles of a similar size.

Work with constrained images

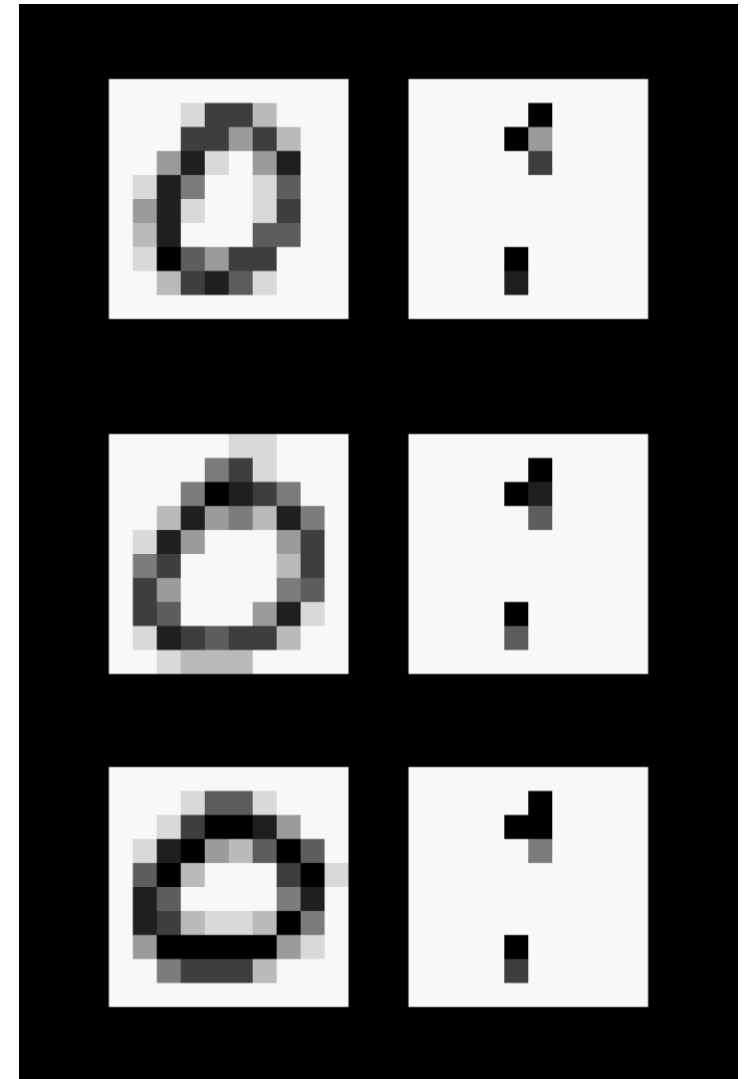


Problem:

- Not achieving significant output.
- The model for image 0 is converging more towards 1 only.

Solution I tried next:

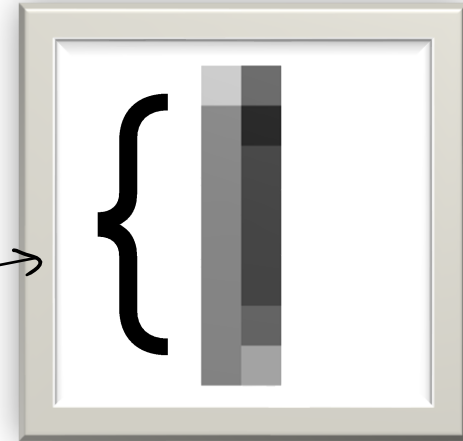
- Adjusted the Hebbian learning threshold for weights close to white to obtain clearer weights.



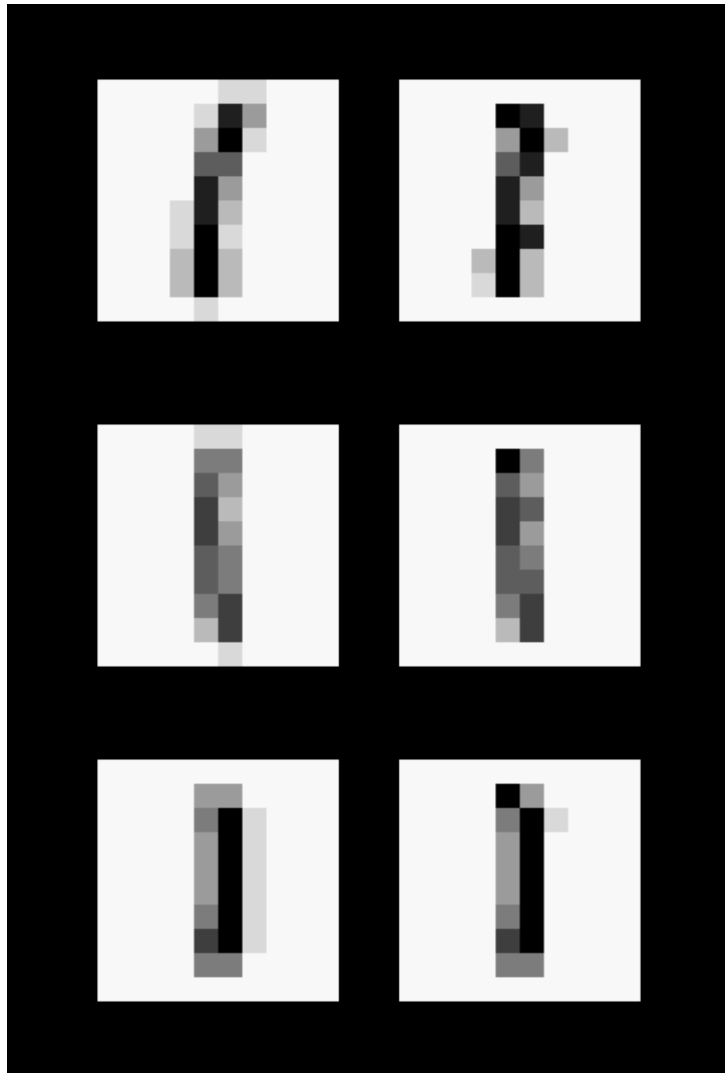
Training weights with a high threshold

- Previously, the threshold for Hebbian learning was 128
i.e. if grayscale value < 128, it is 1 else -1
- It was not strengthening weights with less grey value, also causing Negative impact on weight training.
- Now the new threshold is 220 for training

Ignored



Results with the new threshold.



Success:

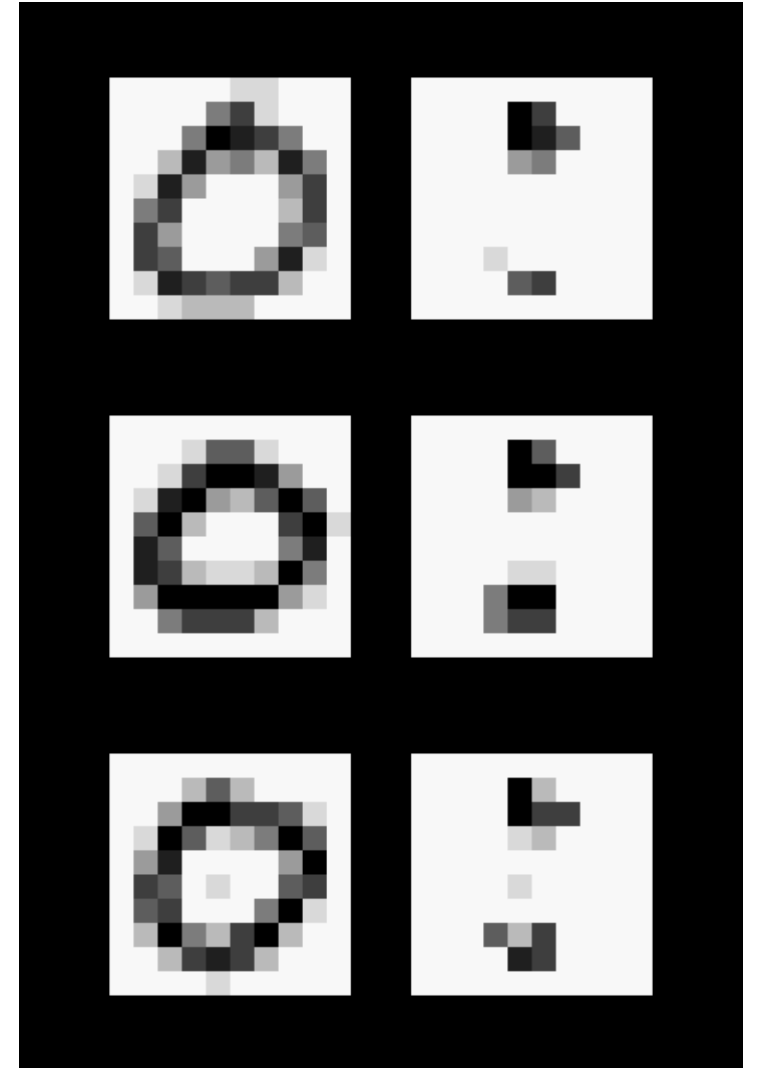
- Obtained significant output images for the digit 1. 😊

Problem:

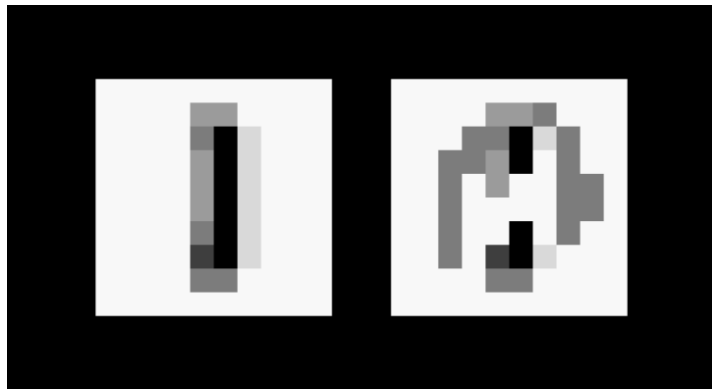
- Not receiving images for the digit 0; they remain as just the intersection of 1 and 0.

Solution Attempted:

- Used double the number of images for training the digit 0 in order to bias the weights towards 0.



Results with bias weights

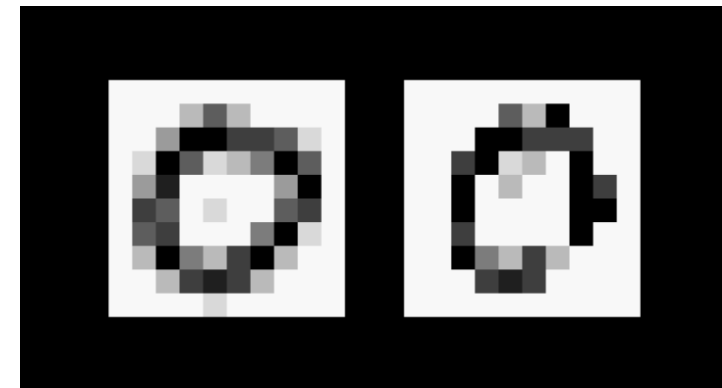
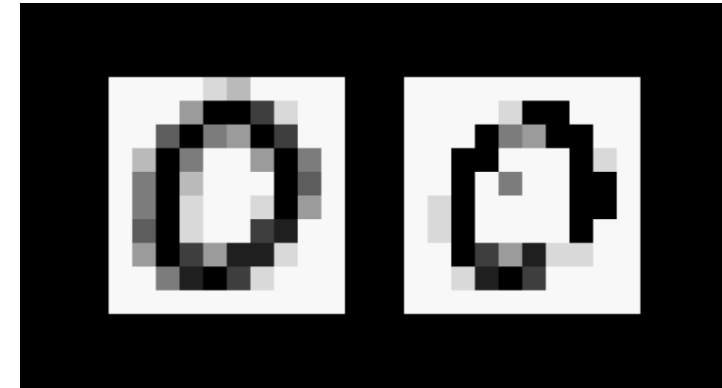


Problem:-

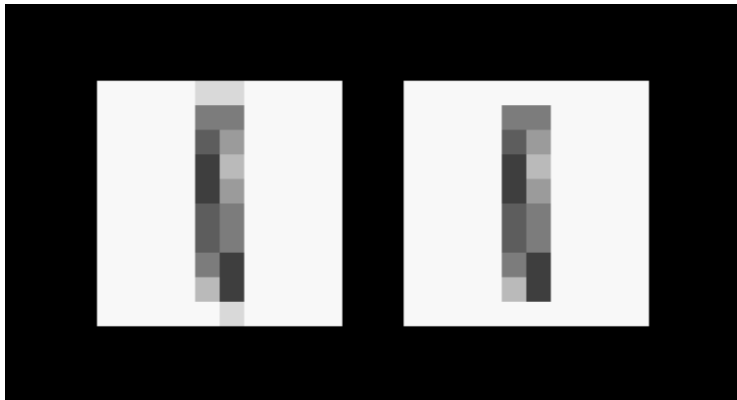
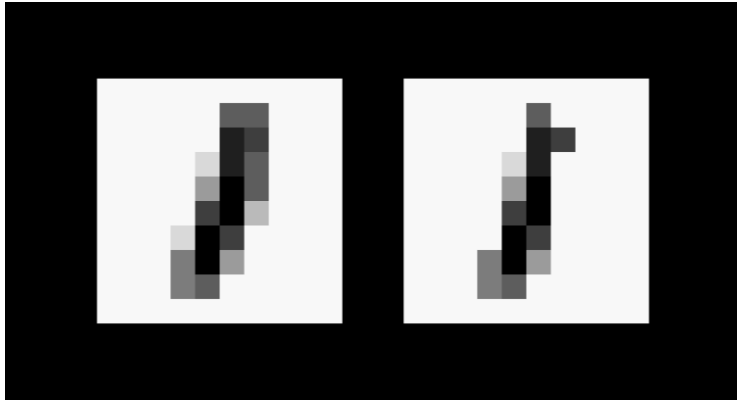
- Got significant image of 0 but destroyed image of 1. 😄 😬

Solution I tried next:-

- Tried with a larger number of training datasets to get stronger weights.



work with a larger dataset

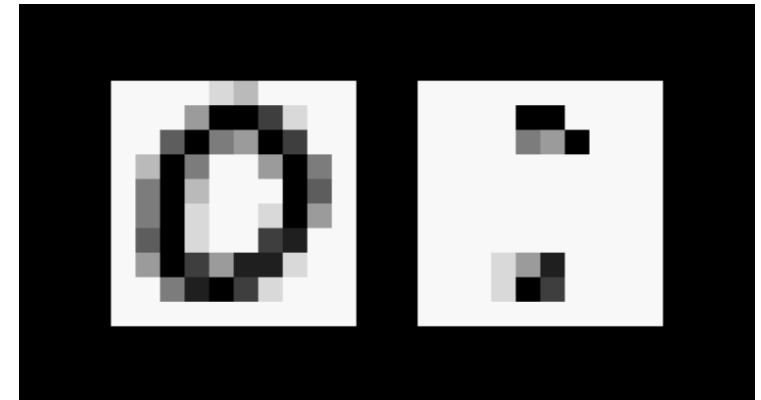
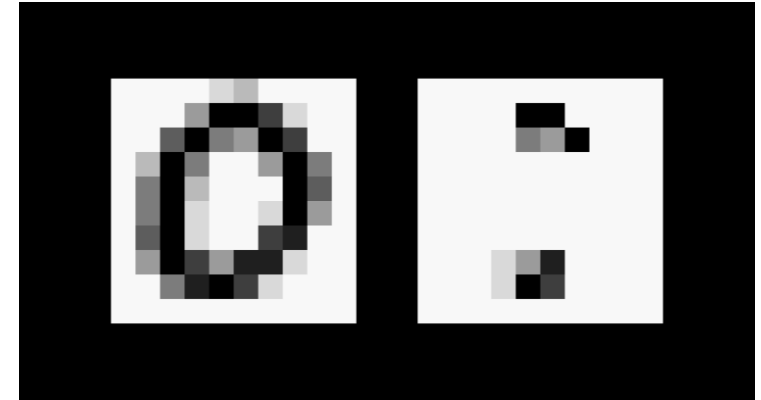


Problem:

- Not receiving images for the digit 0; they remain as just the intersection of 1 and 0.

Solution I tried:-

Now I decided to train the weights with only 1 image per digit multiple times.

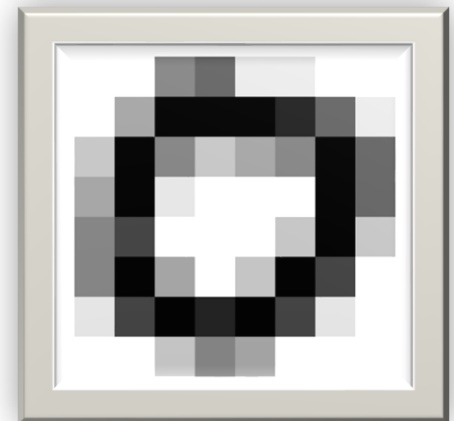
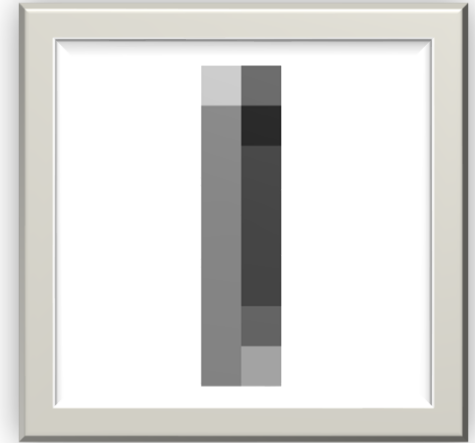


Training weights with only 2 images

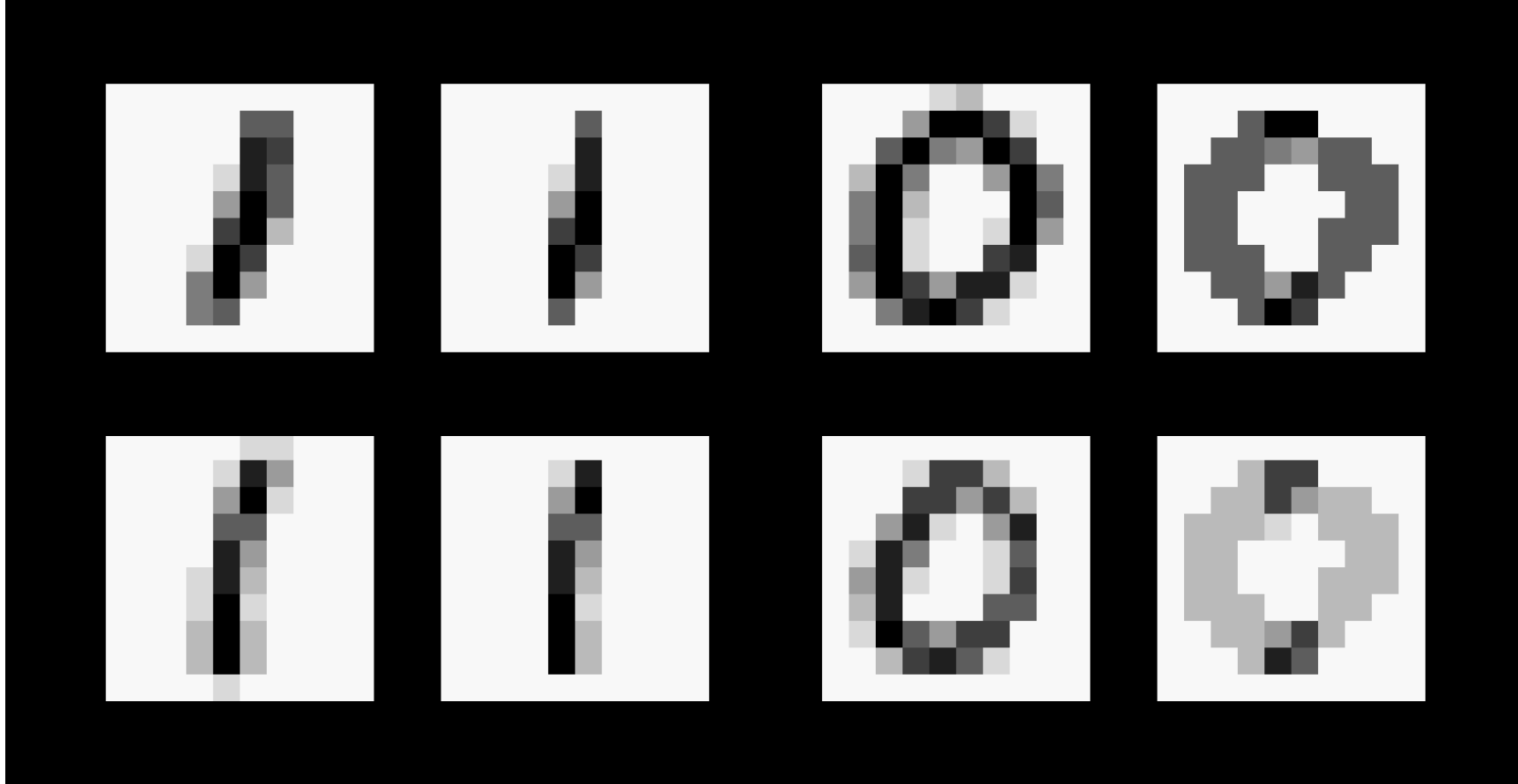
- I have trained weights with only 1 image per digit.
- I have used these adjacent images 15 times for training data.

Thought process:-

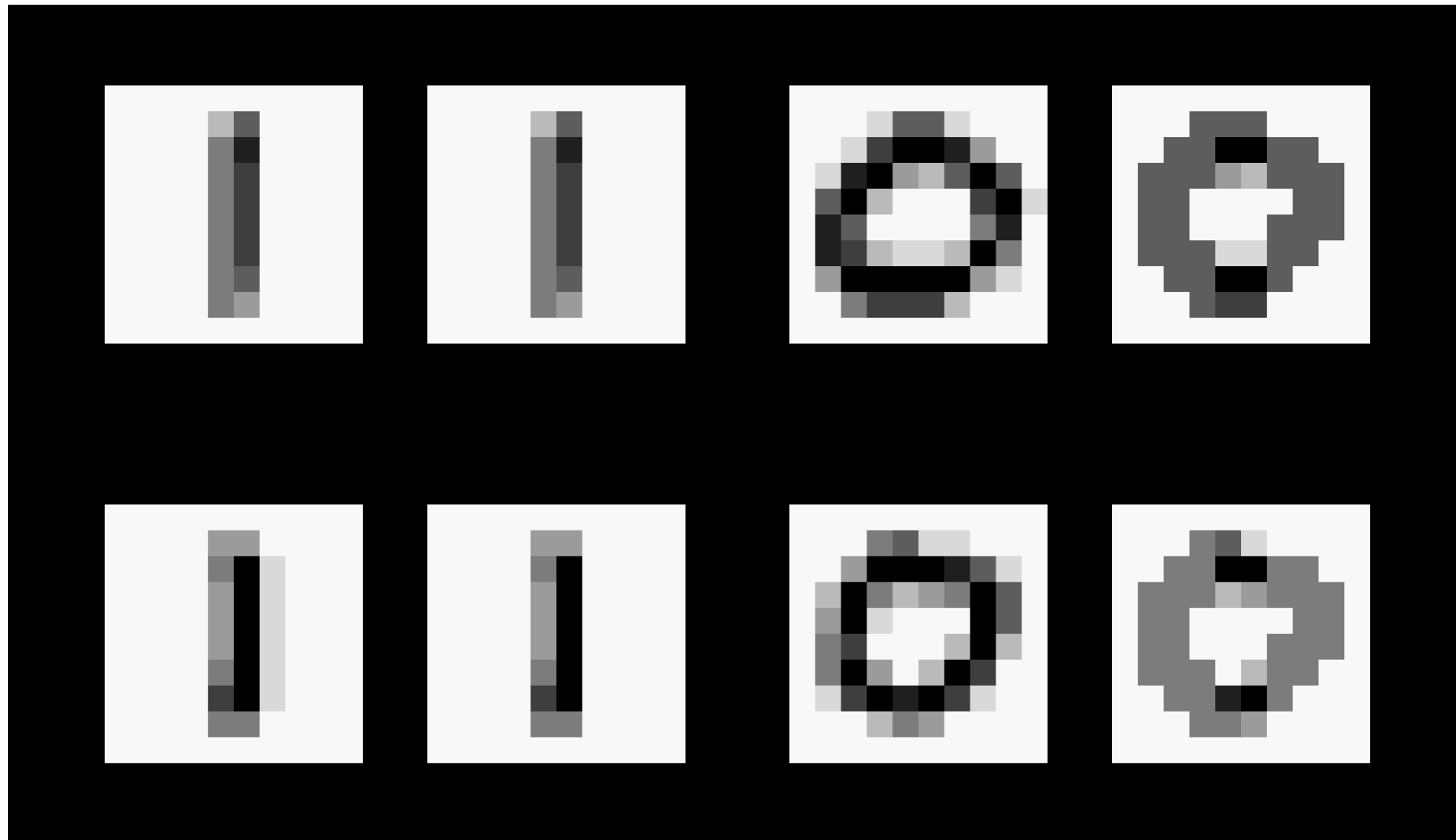
As a larger dataset is not working so let's try with a small one with just only 1 image.



Results:-



More good results:-



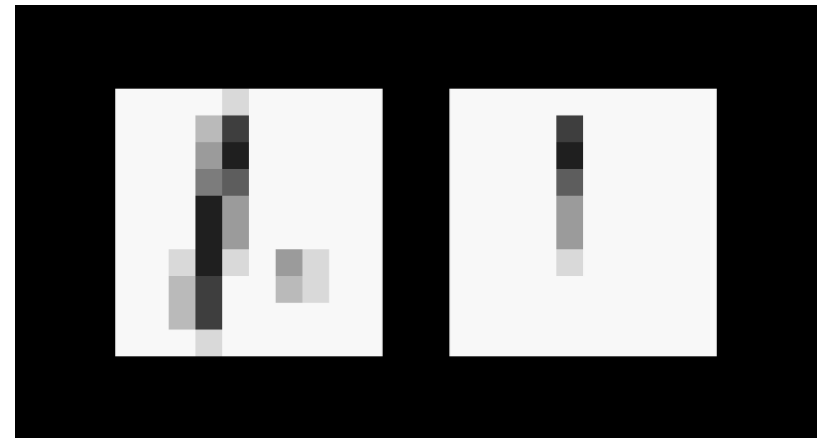
Result analysis

What I got:-

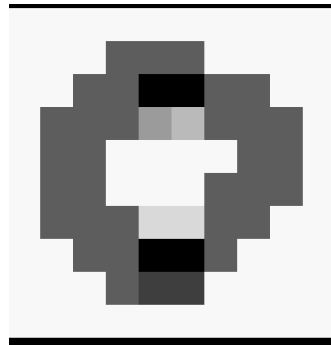
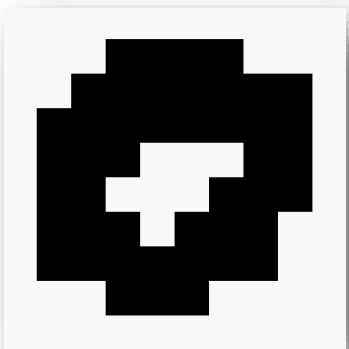
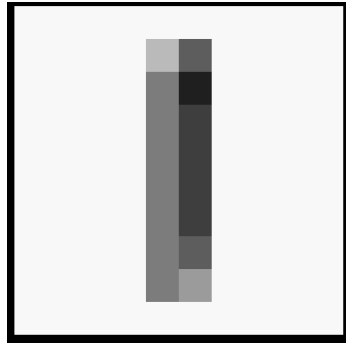
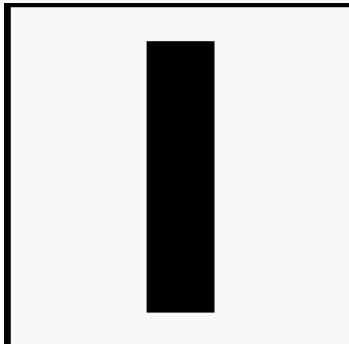
- ✓ Got satisfactory output images for 0,1 for the majority of test images.

Problems:-

- ! Still not converging to a constant image for all input.
- ! These outputs are for just constrained images only.
- ! Not getting a good dark image (compared to the results achieved in previous work).



Hamming distance approach for getting digit output



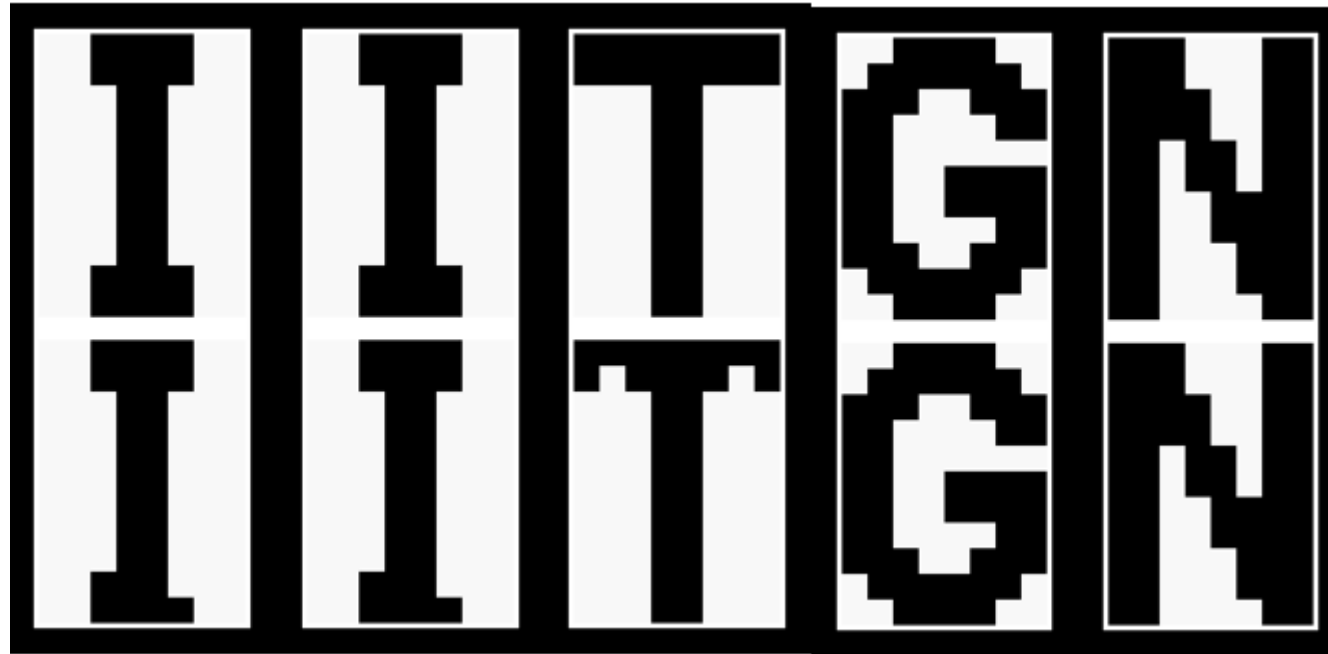
- When the phase of all oscillators converges to a constant, we compare them with the base image.
- By comparing each pixel with the output image pixel, we get the Hamming distance.
- Get the output number based on the minimum hamming distance between 2.

Work with letter recognition

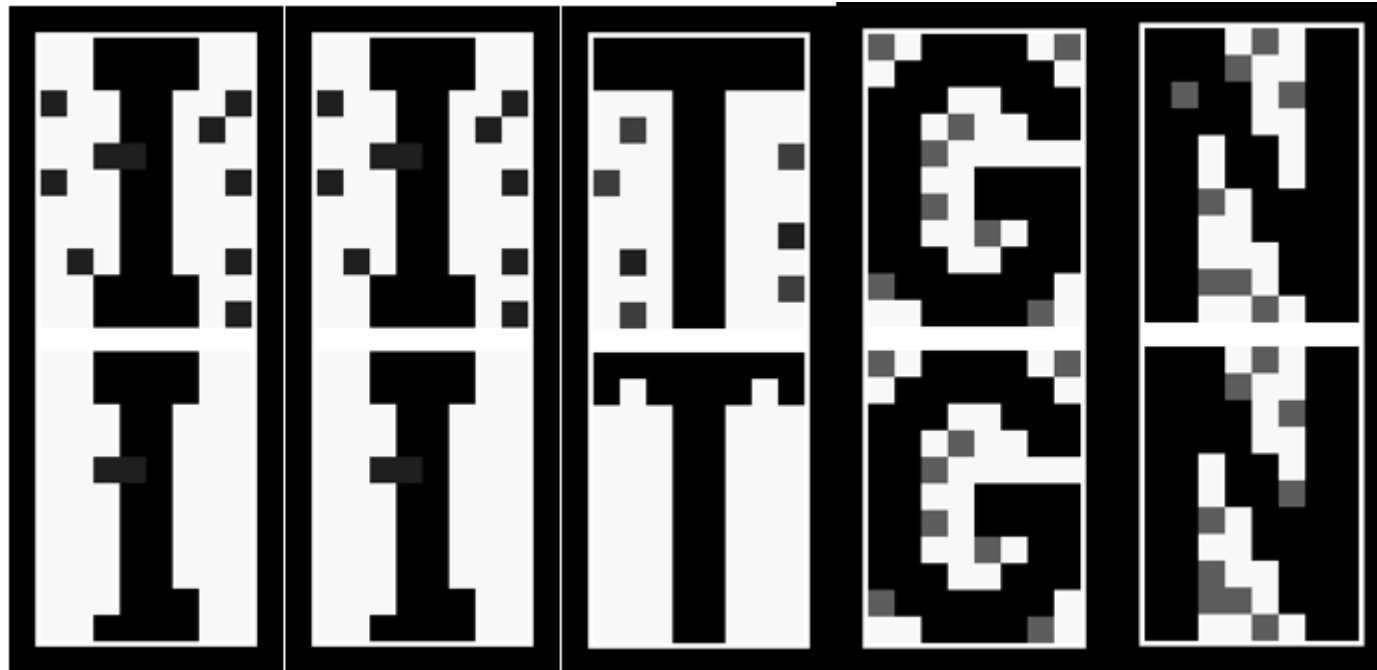
- I have started working with 4 letters 1st 'I' 'T','G','N'



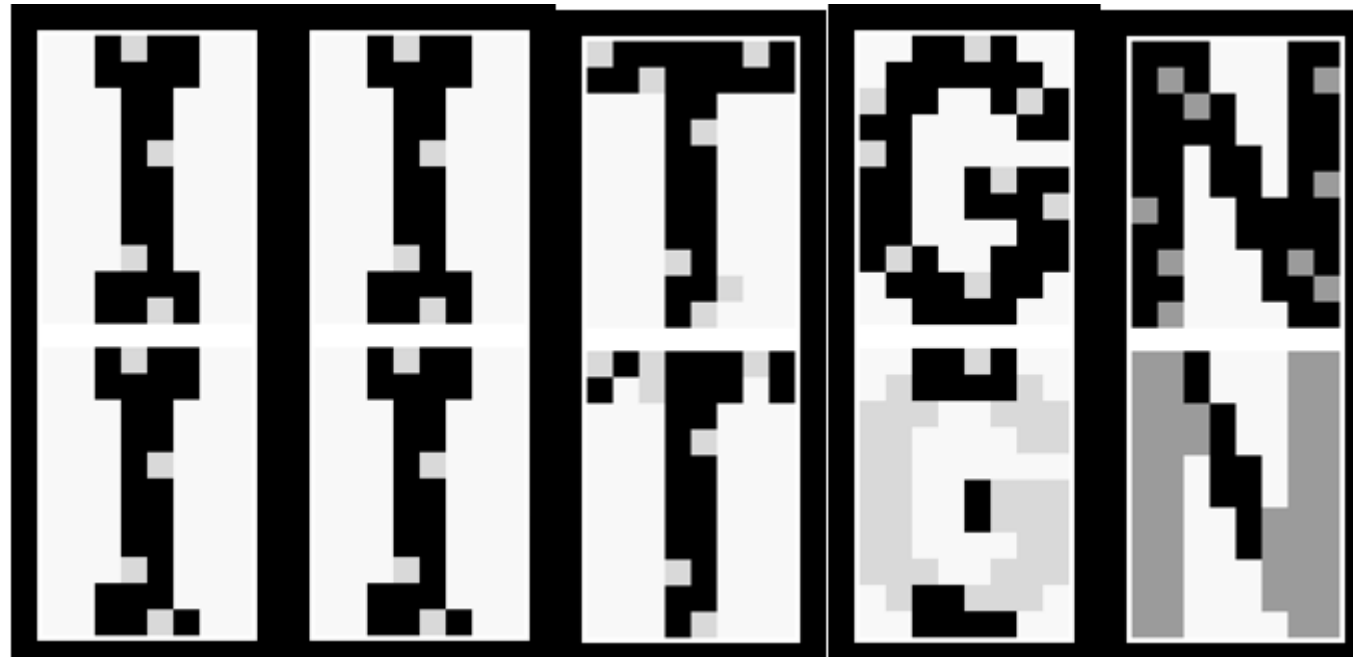
OUTPUT FROM BASE IMAGES



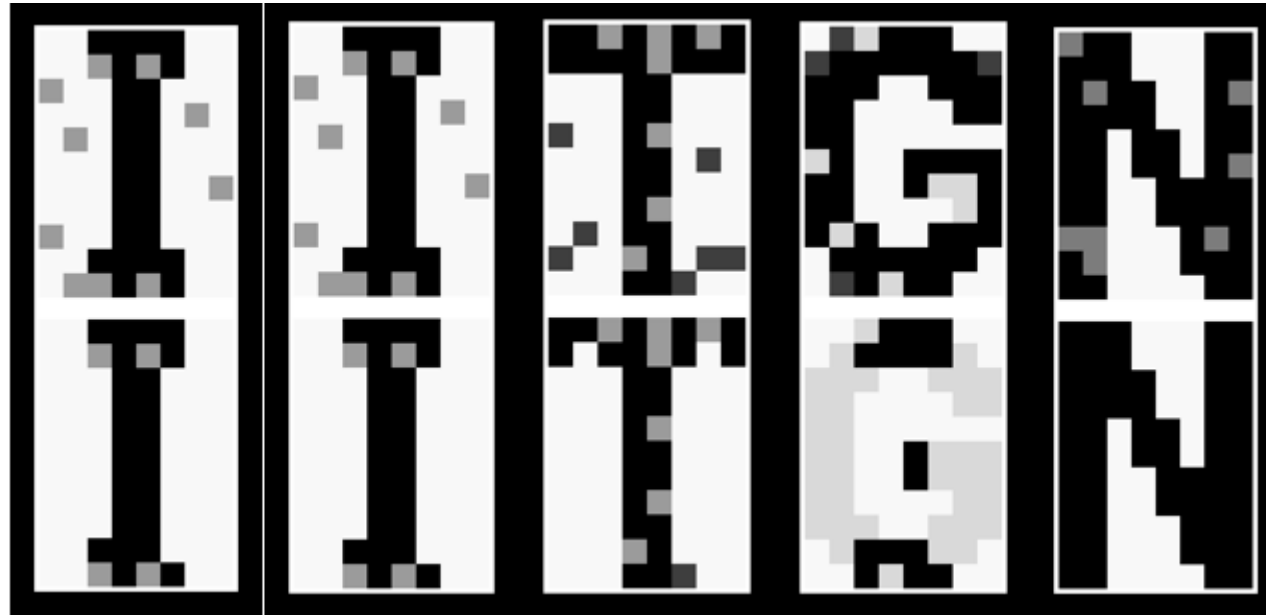
OUTPUT FROM IMAGES WITH NOISE



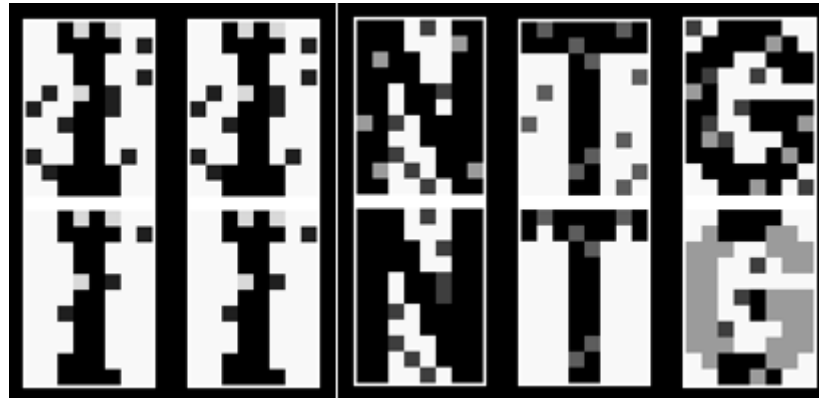
OUTPUT FROM IMAGES WITH NOISE



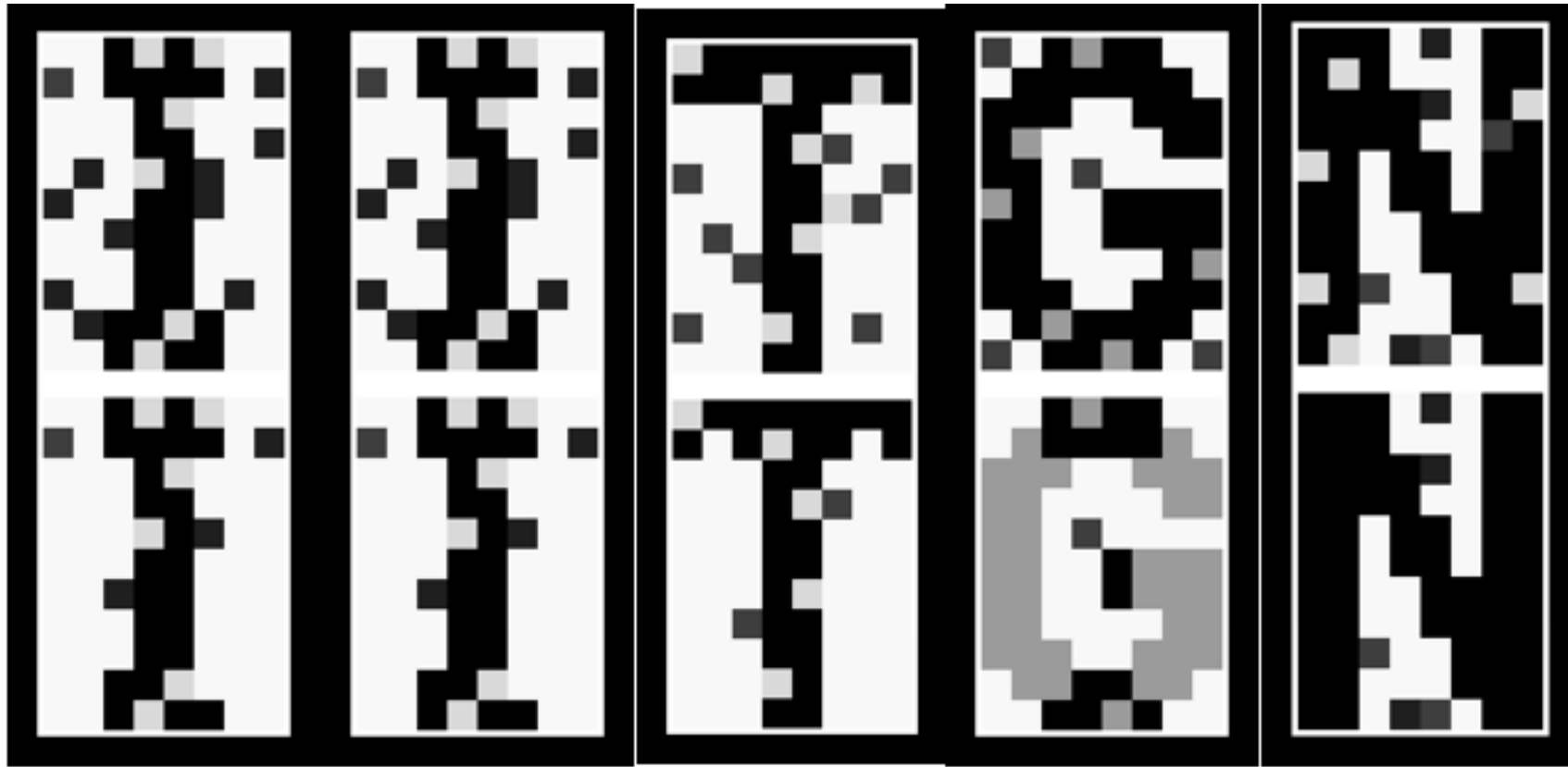
OUTPUT FROM IMAGES WITH NOISE



OUTPUT FROM IMAGES WITH NOISE



OUTPUT FROM IMAGES WITH NOISE



analysis

- Not getting a perfect base image for some letter outputs.
- Added noise remains as it is
- letter recognition is successful for all cases (by the Hamming distance method)

Thank you !!